

SUGARBAYWEB

Flujo de ImageKit para imagenes reales de tienda

Resumen del proceso para conectar ImageKit, subir una imagen, guardar su URL en la base de datos y mostrarla en una tarjeta de producto. Fecha: 28 de abril de 2026.

1. Idea principal

En este proyecto, ImageKit no sustituye a la base de datos. ImageKit guarda el archivo de imagen; Neon/PostgreSQL guarda los datos oficiales del producto; Next.js lee esos datos y renderiza la tarjeta.

ImageKit Media Library

Guarda el archivo real, por ejemplo:

```
https://ik.imagekit.io/gq1enkszp/fotos/camiseta_black_neon?updatedAt=1777406425075
```



Base de datos

Guarda la referencia a la imagen en `ProductImage.imageUrl`, asociada a un `Product`.



lib/repositories/store.ts

Consulta el producto y convierte la primera imagen en `coverImageUrl`.



components/store/store-product-card.tsx

Muestra `coverImageUrl` dentro del componente `<Image />` de Next.js.

Regla practica: para crear otra tarjeta de tienda no se duplica el componente. Se crea otro producto en la base de datos y se crea otra fila en `ProductImage` con la URL de ImageKit.

2. Conexion con ImageKit

Para mostrar imagenes publicas de ImageKit, este proyecto solo necesita el endpoint publico en `.env`.

```
NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT=https://ik.imagekit.io/gq1enkszp
IMAGEKIT_PUBLIC_KEY=""
IMAGEKIT_PRIVATE_KEY=""
```

`NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` se puede usar en cliente y servidor porque empieza por `NEXT_PUBLIC_`. Las claves `IMAGEKIT_PUBLIC_KEY` e `IMAGEKIT_PRIVATE_KEY` se reservan para subidas desde la propia aplicacion.

La private key no debe meterse en componentes de cliente ni subirse a repositorios publicos. Si algun dia se implementa un panel admin para subir imagenes desde la web, la firma debe generarse en servidor.

3. Subir la foto a ImageKit

1. Entrar en el dashboard de ImageKit.
2. Ir a **Media Library**.
3. Crear una carpeta, por ejemplo `fotos` o `products`.
4. Subir la imagen de la camiseta.
5. Copiar la URL publica generada por ImageKit.

Ejemplo real que has usado:

```
https://ik.imagekit.io/gqlenkszp/fotos/camiseta_black_neon?updatedAt=1777406425075
```

Tambien puedes guardar solo el path si el endpoint ya esta en `.env`:

```
fotos/camiseta_black_neon?updatedAt=1777406425075
```

4. Donde se guarda la URL en datos reales

Para datos oficiales, la URL de la imagen debe guardarse en la base de datos. En este proyecto, la tabla relevante es `ProductImage`.

Modelo simplificado

```
Product
- id
- name
- slug
- basePrice
- images: ProductImage[]

ProductImage
- id
- productId
- imageUrl
- altText
- isPrimary
- sortOrder
```

Ejemplo de datos oficiales

```
Product
id: "producto_123"
name: "Sugarbay Tour Tee Black"
```

```
slug: "sugarbay-tour-tee-black"
basePrice: 31.90
isPublished: true

ProductImage
productId: "producto_123"
imageUrl: "https://ik.imagekit.io/gq1enkszp/fotos/camiseta_black_neon?
updatedAt=1777406425075"
altText: "Camiseta negra Sugarbay Neon"
isPrimary: true
sortOrder: 1
```

Con Prisma Studio se puede editar esto visualmente:

```
npm.cmd exec prisma studio
```

En Studio: abrir `Product`, localizar el producto, copiar su `id`, abrir `ProductImage`, buscar la fila con ese `productId` y actualizar `imageUrl`.

No conviene depender de `prisma/seed.ts` como fuente principal de datos oficiales. El seed sirve para cargar datos iniciales o demo. La fuente real debe ser la base de datos.

5. Como viaja la imagen por el codigo

Paso A: el repositorio consulta productos e imagenes

En `lib/repositories/store.ts`, la consulta pide las imagenes del producto ordenadas por prioridad:

```
images: {
  select: {
    imageUrl: true,
    isPrimary: true,
    sortOrder: true,
  },
  orderBy: [
    { isPrimary: "desc" },
    { sortOrder: "asc" },
  ],
  take: 1,
}
```

Esto significa: de todas las imagenes del producto, dame primero la marcada como principal; si hay varias, ordena por `sortOrder`; usa solo una.

Paso B: se crea el dato que necesita la tarjeta

En la funcion que transforma el registro de base de datos a un producto de tienda, se crea `coverImageUrl`:

```
coverImageUrl: record.images[0]?.imageUrl ?? null,
```

Traduccion: si el producto tiene una imagen, usa la primera. Si no tiene imagen, usa `null`.

Paso C: la pagina pasa cada producto a la tarjeta

En `app/store/page.tsx`, la tienda recorre todos los productos y crea una tarjeta para cada uno:

```
{catalog.products.map((product) => (
  <StoreProductCard key={product.id} product={product} />
))}
```

Aqui no se decide la URL. La pagina solo entrega el objeto `product` a la tarjeta.

Paso D: la tarjeta renderiza la imagen

En `components/store/store-product-card.tsx` se usa `product.coverImageUrl` como origen de la imagen:

```
<Image
  src={resolveImageUrl(product.coverImageUrl)}
  alt={product.name}
  fill
  className="object-cover"
  sizes="(max-width: 768px) 100vw, (max-width: 1280px) 50vw, 33vw"
/>
```

La tarjeta no sabe si la imagen viene de ImageKit, de otra URL externa o de un path relativo. Solo llama a `resolveImageUrl()`.

6. Que hace resolveImageUrl()

En `lib/services/imagekit.ts`, esta funcion normaliza el valor que viene de la base de datos.

```
export function resolveImageUrl(path?: string | null): string {
  if (!path) return "/next.svg";
  if (path.startsWith("http://") || path.startsWith("https://")) return path;

  if (!DEFAULT_IMAGEKIT_ENDPOINT) {
    return `/${stripSlashes(path)}`;
  }

  return `${DEFAULT_IMAGEKIT_ENDPOINT.replace(/\/+$/, "")}/${stripSlashes(path)}`;
}
```

Caso 1: guardas la URL completa

```
ProductImage.imageUrl =
"https://ik.imagekit.io/gq1enkszp/fotos/camiseta_black_neon?updatedAt=1777406425075"

resolveImageUrl(...) devuelve la misma URL.
```

Caso 2: guardas solo el path

```
NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT =
"https://ik.imagekit.io/gq1enkszp"

ProductImage.imageUrl =
"fotos/camiseta_black_neon?updatedAt=1777406425075"

resolveImageUrl(...) devuelve:
"https://ik.imagekit.io/gq1enkszp/fotos/camiseta_black_neon?updatedAt=1777406425075"
```

Para este proyecto, cualquiera de las dos opciones funciona. Guardar el path es mas limpio si todas las imagenes vienen de la misma cuenta de ImageKit.

7. Por que Next.js acepta esa imagen remota

El componente `next/image` necesita que el host remoto este permitido en `next.config.ts`. Tu proyecto ya lo prepara:

```
const imageKitHostnames = new Set<string>(["ik.imagekit.io"]);

const nextConfig: NextConfig = {
  images: {
    remotePatterns: Array.from(imageKitHostnames).map((hostname) => ({
      protocol: "https",
      hostname,
      pathname: "/*",
    })),
  },
};
```

Esto permite que `<Image />` optimice imagenes servidas desde `https://ik.imagekit.io/...`.

8. Ejemplo completo: crear un producto con imagen

Este ejemplo muestra la idea programatica. En una aplicacion real, este codigo estaria detras de un panel admin protegido o se haria desde Prisma Studio.

```
const product = await prisma.product.create({
  data: {
    categoryId: tshirtsCategory.id,
    name: "Camiseta Black Neon",
    slug: "camiseta-black-neon",
    description: "Camiseta oficial Sugarbay con arte neon.",
    productType: ProductType.APPAREL,
    basePrice: 31.9,
    currency: "EUR",
    isPublished: true,
    images: {
      create: [
        {
          imageUrl:
            "https://ik.imagekit.io/gq1lenkszp/fotos/camiseta_black_neon?updatedAt=1777406425075",
          altText: "Camiseta negra Sugarbay Neon",
          isPrimary: true,
          sortOrder: 1,
        },
      ],
    },
  },
});
```

Ejemplo para actualizar la imagen principal de un producto existente

```
const product = await prisma.product.findUnique({
  where: { slug: "sugarbay-tour-tee-black" },
  select: { id: true },
});

if (!product) {
  throw new Error("Producto no encontrado");
}

await prisma.productImage.updateMany({
  where: {
    productId: product.id,
    isPrimary: true,
  },
  data: {
    imageUrl:
      "https://ik.imagekit.io/gq1lenkszp/fotos/camiseta_black_neon?updatedAt=1777406425075",
    altText: "Camiseta negra Sugarbay Neon",
    sortOrder: 1,
  },
});
```

9. Cuando se usan IMAGEKIT_PUBLIC_KEY e IMAGEKIT_PRIVATE_KEY

Ahora mismo, como subes imagenes manualmente desde el dashboard de ImageKit, no necesitas esas claves para mostrar imagenes.

Se usarian si quieres crear un panel dentro de Sugarbay para subir imagenes desde la web. El flujo seria:

1. El usuario admin selecciona un archivo en la web.
2. El cliente pide parametros de autenticacion a `/api/imagekit/auth`.
3. El servidor genera `token`, `signature` y `expire` usando la private key.
4. El cliente sube el archivo a ImageKit con esos parametros.
5. La app guarda la URL devuelta por ImageKit en `ProductImage.imageUrl`.

Tu proyecto ya tiene preparada esta parte de servidor:

```
const imageKit = getImageKitClient();
const authParams = imageKit.helper.getAuthenticationParameters();
return Response.json(authParams);
```

Pero falta una interfaz admin que use ese endpoint. Por eso, en la fase actual lo mas simple es: subir en ImageKit manualmente y guardar la URL en la base de datos.

10. Checklist recomendado

1. Configurar `NEXT_PUBLIC_IMAGEKIT_URL_ENDPOINT` con tu endpoint real.
2. Subir cada imagen oficial a ImageKit Media Library.
3. Copiar la URL o el path del archivo.
4. Crear o editar el producto real en la base de datos.
5. Crear o editar su fila en `ProductImage`.
6. Marcar la imagen principal con `isPrimary: true`.
7. Usar `sortOrder: 1` para la imagen principal.
8. Refrescar `/store` y comprobar que aparece la tarjeta.

11. Fuentes

La explicacion del flujo interno sale de los archivos locales del proyecto:

- `components/store/store-product-card.tsx`
- `lib/repositories/store.ts`
- `lib/services/imagekit.ts`
- `app/api/imagekit/auth/route.ts`
- `next.config.ts`
- `prisma/schema.prisma`

Documentacion oficial consultada:

- ImageKit - Upload assets: <https://imagekit.io/docs/dam/upload-assets>
- ImageKit - API keys: <https://imagekit.io/docs/api-keys>
- ImageKit - React SDK y parametros de autenticacion de subida: <https://imagekit.io/docs/integration/react>

- Next.js - Image Component, remote images y remotePatterns: <https://nextjs.org/docs/app/api-reference/components/image>
- Prisma - Prisma Studio: <https://www.prisma.io/docs/v6/orm/tools/prisma-studio>
- Prisma - CRUD con Prisma Client: <https://docs.prisma.io/docs/orm/prisma-client/queries/crud>
- Prisma - Relaciones y nested writes: <https://docs.prisma.io/docs/v6/orm/prisma-schema/data-model/relations>